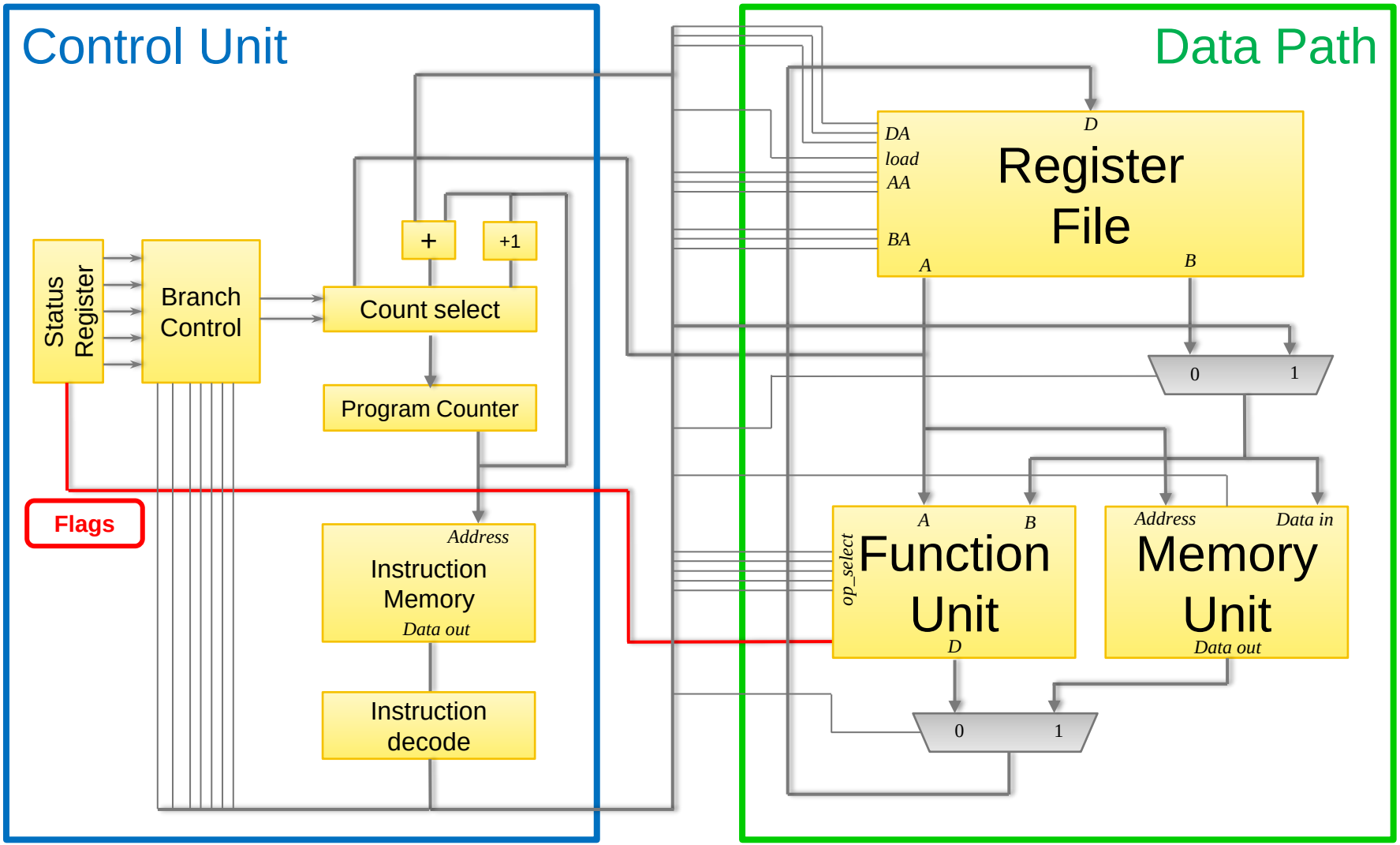
Review – basic instruction set architectures

ADVANCED COMPUTING ARCHITECTURES

Typical processor architecture



Instruction Set Principles

Instruction Set Architectures (ISA) fundamentals

Instruction classification

Addressing modes

Classifying ISA types

Instruction Set Architectures (ISA)

Number and complexity of instructions

Complex Instruction Set Computers (CISC): large number of powerful instructions with very different execution times

+ Lower "semantic gap"

- Very complex Control Unit

Reduced Instruction Set Computers (RISC): small number of simple instructions, making elementary operations within similar execution times

+ Simpler and more modular organization

- Larger machine-level programs

Internal storage architecture and addressing modes

- Stack architectures
- Accumulator architectures
- General-purpose register (GPR) architectures

Register-memory

Register-register (or load-store)

Instruction definition

A binary or machine-level instruction is a multiple-field binary string to be decoded by the control unit of the processor.

It represents the interface between HW and SW.

In general, an instruction consists of:

- An opcode field describing the operation that has to be performed
- Some address fields indicating where the input operands are located and where the result should be stored
- (in some old architectures) an equal number of address qualifier fields telling how to access the operand reported in the corresponding address field

Instruction Set Architecture

The instruction set architecture (ISA) includes all the rules describing the functional specifications of a given architecture

Each ISA is characterized by:

- Different classes of instruction operators
- Number of operands
- Different addressing modes, i.e., different mechanisms to locate and to access the operands for a given instruction

Classes of instruction operators

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiple, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

All computers provide a full set of operations for the first 3 categories

System functions in the instruction set vary widely among architectures

The amount of support for the last four categories may vary largely

- Floating-point instructions are sometimes part of an optional instruction set
- Decimal and string were primitives in old systems, but they are no longer used in nowadays processors
- Graphic and multimedia instructions typically operate on many smaller data items in parallel; e.g., performing eight 8-bit additions on two 64-bit operands

An example justifying RISC philosophy

Rank	80x86 instruction	Integer average (% total executed)
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-register	4%
9	call	1%
10	return	1%
Total		96%

Statistics related to 5 programs of the SPEC suite running on a x86 processor

- 96% of the executed x86 instructions are very simple.

In accordance with Amdahl's law, it is better to improve such instructions, while sacrificing the remaining 4%

- RISC architecture is more effective

In fact, recent x86 architectures have a RISC core, while keeping the same traditional CISC structure for binary software compatibility

Arithmetic instructions

Arithmetic and logic operations

- Add, Sub, Mul, And, Or, Not

Shifting: instructions for shifting the operand by an arbitrary number of bits

- Arithmetic shift: left shift: zero into LSB; right shift: sign extended into MSB
- Logical shift: left shift: zero into LSB; right shift: zero into MSB;
- Circular shifting: rotate left: MSB → LSB; rotate right: LSB → MSB

Comparison

- ALU outputs provide information about the results of arithmetic operations (Zero bit, Sign bit, Overflow bit)
- Such bits can be used by compare instructions

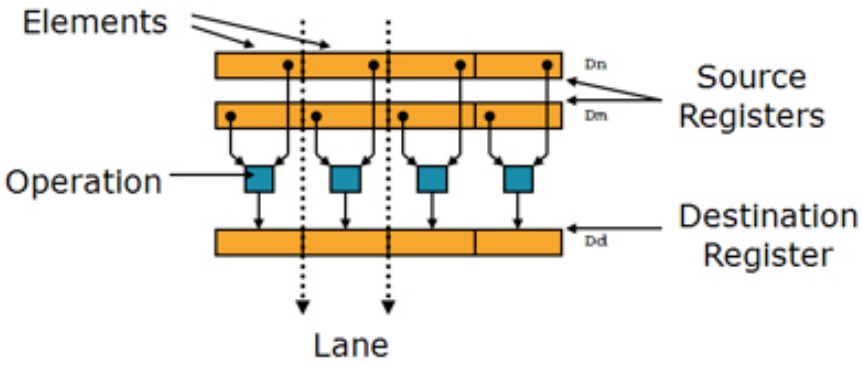
Multimedia instructions

The data for multimedia operations is often narrower than the 64-bit data word of modern processors

- Rather than wasting the 64-bit ALUs when operating on 32-bit, 16-bit, or 8-bit integers, multimedia instructions operate on several narrower data items in parallel
- A partitioned add operation on 16-bit data with a 64-bit ALU performs four 16-bit adds in a single clock cycle.
- The extra hardware cost is simply to prevent carries between the four 16-bit partitions of the ALU.

A similar approach can be also used with floating-point operations (2 single- precision operations on a double-precision ALU)

These operations are commonly called Single-Instruction Multiple Data (SIMD) or vector instructions



Data transfer instructions

Load from memory into register

Store register into memory

Movements between registers

Movements between different memories

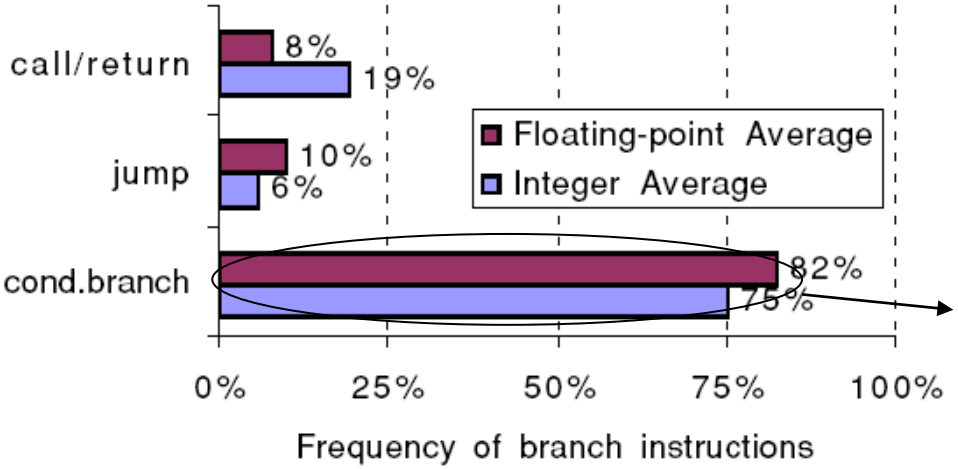
Pushing and popping the stack

Sometimes also I/O operations through specific ports

Instructions for control flow

- ***Jumps*** **and conditional** ***branches***
  - The Jump is an unconditional branch
  - Conditional branch "taken" only when a certain condition is fulfilled
- **Subroutine** ***calls*** **and** ***returns*** **from subroutine**
- ***Trap*** **and** ***wait*** **instruction for temporarily halting program execution**
- ***Nops*****, i.e. no operation (it can be used as a delay)**
- **Special** ***repeat*** **instructions (only in DSPs)**

Example: Control flow instructions in SPEC applications running on an Alpha platform



Conditional branches are critical for performance!

Conditional branch options

Since most changes in control flow are branches, deciding how to specify the branch condition is important.

Usually the 3 following conditions are used:

Name	Examples	How condition is tested	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Special bits are set by ALU operations, possibly under program control.	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructions since they pass information from one instruction to a branch.
Condition register	Alpha, MIPS	Tests arbitrary register with the result of a comparison.	Simple.	Uses up a register.
Compare and branch	PA-RISC, VAX	Compare is part of the branch. Often compare is limited to subset.	One instruction rather than two for a branch.	May be too much work per instruction for pipelined execution.

Given that a large number of the comparisons is with zero, some architectures choose to treat these comparisons as special cases, especially if a compare and branch instruction is being used.

System and other special instructions

Many other kinds of instructions may exist for system or special purposes

- Virtual memory management
- Operating system calls
- Masking and setting interrupts
- Configuring and resetting on chip peripherals
- Bit manipulation instructions
  - Bit set, bit clear, bit toggle, and bit test
  - Branch-if-bit-set, branch-if-bit-clear
- Iterative division
- Square root
- Other architecture-specific instructions

ADDRESSING MODES

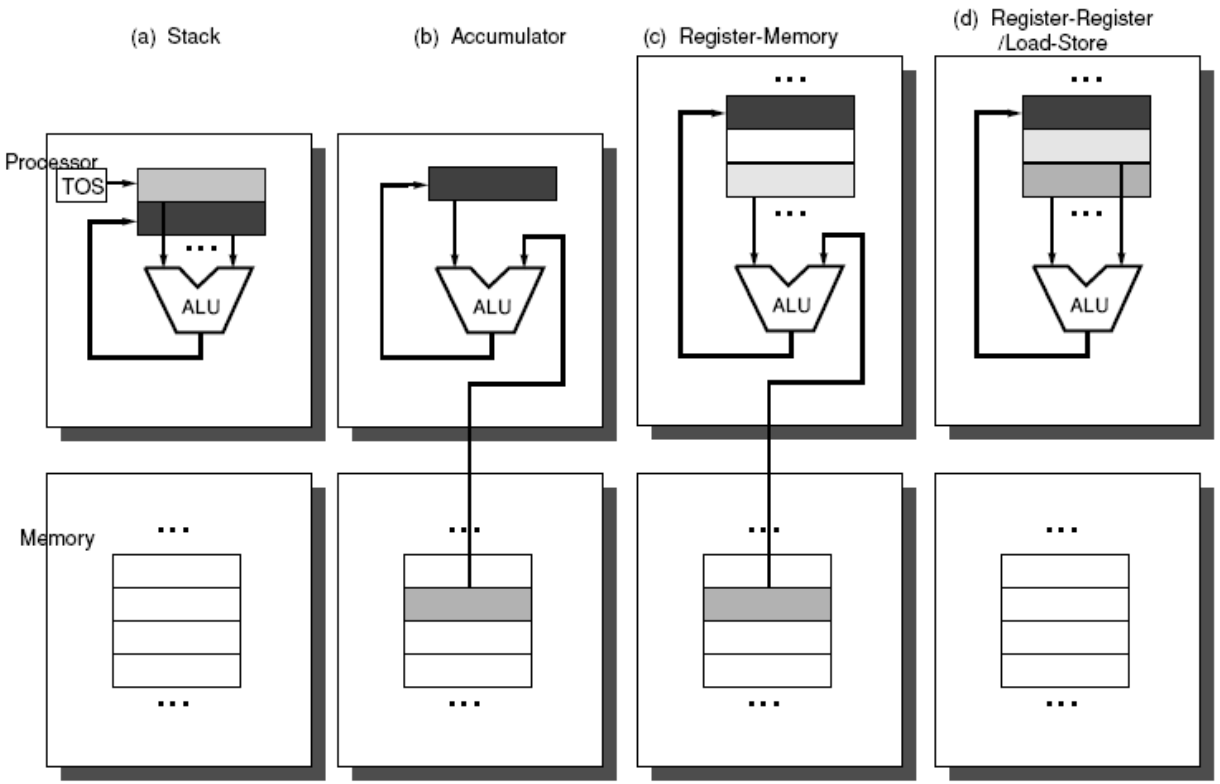
Classifying operand access modes

Stack: operands are on the top of the stack

Accumulator: one operand in memory, the other and the result in the accumulator

Register-memory: one operand in memory, the other and the result in two registers of the register file

Register-register: both operands have to be placed from memory into registers. The result is also stored in a register



Example: C = A + B

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

Addressing modes

Usually, instructions have 2 or 3 operands. In case of two operands, one of them represents also the location where the result should be stored

Immediate addressing

- The operand is a constant specified within the instruction

Example: ADD R4, R2, #3

- Small data words may be included into instruction word typically half of the instruction word width
- longer data words may be stored into word following the instruction → requires two access to program memory (instruction and operand)

Addressing modes

Register addressing

- Required operands can be found in a specified register

Example: DSUB R3, R1, R2

- In MIPS64, value in register R1 is subtracted from value in register R2 and the result is stored in R3

Addressing modes

Register indirect

- The operand is in the memory location pointed by a register

Example: ADD R3, (R2)

- Add the content of R3 to the value in memory at the address pointed to by R2. Store the result in R3

Register indirect addressing modes are important because they allow natural and automatic modification of pointers by modifying the address value.

Addressing modes

Register indirect, autoincrement

- Autoincrement or to autodecrement the register content (before or after performing the memory access) by a known quantity

Example: ADD R3, (R2)+

- post-increment the value in R2 after the addition

Addressing modes

Register indirect, displacement, indexed

- Increment or decrement the register value by a constant (displacement mode) or by the content of another register (indexed mode)

Example (displacement): ADD EAX, 1000(EBX)

- add the content of the register EAX with the content of the location of memory given by EBX+1000 and place the result into EAX

Example (indexed): ADD R3, (R1 + R2)

- Memory location at address R1 + R2
- Indexed addressing is useful when using the same code for accessing several data arrays

Addressing modes

Memory direct addressing

- The operand is fetched from an explicit address of memory or can be found from a separate data word following the instruction

Example: ADD R1, (1001)

- Add value contained at address 1001 to register R1, and store result in R1
- Small addresses can be encoded into the instruction word, long addresses require a separate data word following the instruction code

Addressing modes

PC-relative addressing

- This is a special type of displacement addressing, in which the considered register is the Program Counter
- This is particularly useful to calculate the destination address of branch instructions, which are known at compile time

ADDRESSING MODES: SUMMARY

Register name

Memory location

Assign operator

Addressing mode	Example instruction	Meaning	When used
Register	Add R4,R3	Regs[R4] ← Regs[R4] + Regs[R3]	When a value is in a register.
Immediate	Add R4,#3	Regs[R4] ← Regs[R4] + 3	For constants.
Displacement	Add R4,100(R1)	Regs[R4] ← Regs[R4] + Mem[100+Regs[R1]]	Accessing local variables (+ simulates register indirect, direct addressing modes)
Register indirect	Add R4,(R1)	Regs[R4] ← Regs[R4] + Mem[Regs[R1]]	Accessing using a pointer or a computed address.
Indexed	Add R3,(R1 + R2)	Regs[R3] ← Regs[R3] + Mem[Regs[R1] + Regs[R2]]	Sometimes useful in array addressing: R1 = base of array; R2 = index amount.
Direct or absolute	Add R1,(1001)	Regs[R1] ← Regs[R1] + Mem[1001]	Sometimes useful for accessing static data; address constant may need to be large.
Memory indirect	Add R1,@(R3)	Regs[R1] ← Regs[R1] + Mem[Mem[Regs[R3]]]	If R3 is the address of a pointer *p*, then mode yields **p*.
Autoincrement	Add R1,(R2)+	Regs[R1] ← Regs[R1] + Mem[Regs[R2]] Regs[R2] ← Regs[R2] + *d*	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, *d*.
Autodecrement	Add R1,-(R2)	Regs[R2] ← Regs[R2] − *d* Regs[R1] ← Regs[R1] + Mem[Regs[R2]]	Same use as autoincrement. Autodecrement/increment can also act as push/pop to implement a stack.
Scaled	Add R1,100(R2)[R3]	Regs[R1] ← Regs[R1] + Mem[100+Regs[R2] + Regs[R3]*d]	Used to index arrays. May be applied to any indexed addressing mode in some computers.

Instruction Encoding

Assembly Language

Symbolic representation of the machine language of a specific processor

Invented to simplify programming

- Converted to machine code by an assembler program

Assembly language consists of instructions or directives

- Instructions are the "readable" version of the machine-level instructions
- Example: **MACR *AR5, *AR6, A, B ;this is a comment**
- Directives (usually starting with "." like in ".data") guide assembling and linking operations, such as reserving a portion of memory, define global and external variables
- Directives do not consume space in memory

Some examples of common directives

.align n → **Cause the next data/code loaded to be at the next higher address with the lower n bits zeroed**

.ascii "string1","...“ → **Store the "strings" listed on the line in memory as a list of characters**

.byte byte1, byte2,… → **Store the bytes listed on the line sequentially in memory**

.data → **Cause the following code and data to be stored in the data area**

.double (.int) number1, → **Store the "numbers" listed on the line sequentially in memory as double-precision floating point (integer) numbers**

.global label → **Make the label available for reference by code found in files loaded after this file**

.text → **Cause the following code and data to be stored in the text (code) area.**

INSTRUCTION ENCODING

Instruction encoding affects:

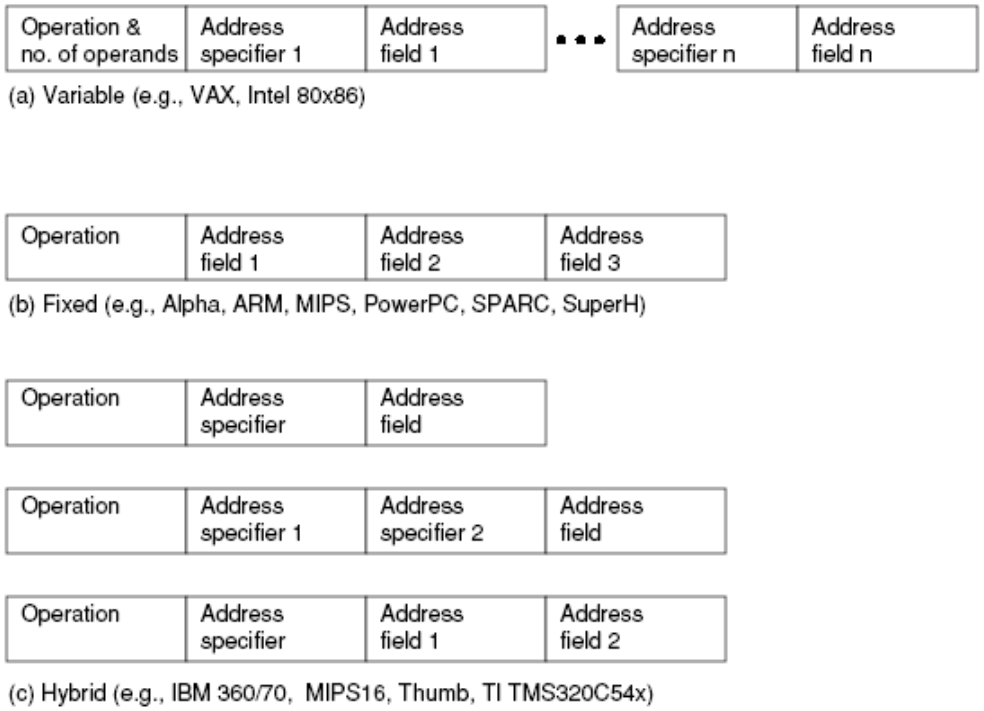
- the size of the compiled programs
- the implementation of the processor, which must decode this representation to quickly find the operation and its operands.

3 main encoding schemes

- **Variable** (in opcode size or in number of operands): suitable when there are many addressing modes and operations. Flexible, but complex decoding logic

- **Fixed:** opcode size and number of operands are fixed. Suitable when there are few addressing modes and operations, but they lead to larger programs

- **Hybrid**: multiple formats specified by the opcode, adding one or two fields to specify the addressing mode or the operand address



DLX INSTRUCTION SET

DLX instructions are 32-bit wide with a 6-bit opcode

The operation encoding is fixed. The instruction formats are of Immediate type, Register type or Jump type (see picture)

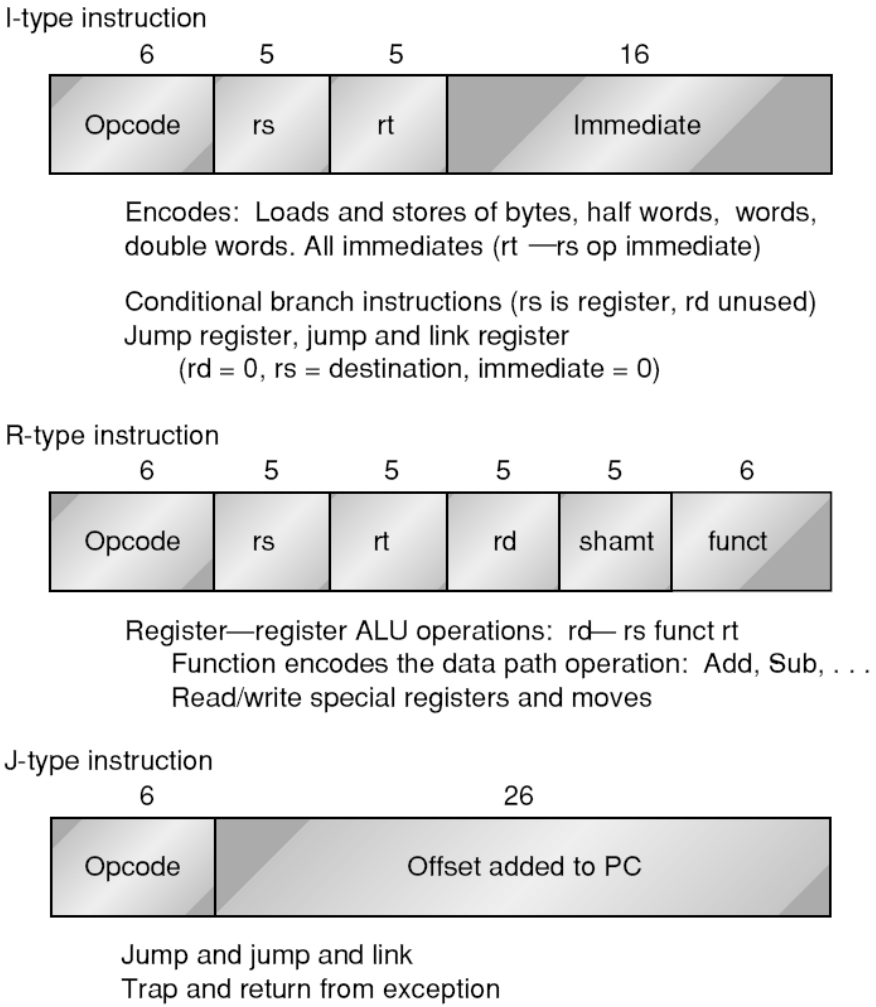
In DLX there are four classes of instructions:

- Data transfer instructions
- Integer arithmetic and logical instructions
- Control instructions
- Floating point instructions

There are only 3 addressing modes: Register, Immediate and Displacement

Register indirect is accomplished simply by placing 0 in the 16-bit displacement field

Direct memory addressing is accomplished by using R0 as the base register



Data Transfer instructions in DLX

Opcode/Mnemonic	*Operands*	*Meaning*
LB	Rd,Adr	Load byte (sign extension)
LBU	Rd,Adr	Load byte (unsigned)
LH	Rd,Adr	Load half-word (sign extension)
LHU	Rd,Adr	Load half-word (unsigned)
LW	**Rd,Adr**	**Load word**
LF	Fd,Adr	Load single-precision Floating point
LD	**Dd,Adr**	**Load double-precision Floating point**
SB	Adr,Rs	Store byte
SH	Adr,Rs	Store half-word
SW	**Adr,Rs**	**Store word**
SF	Adr,Fs	Store single-precision Floating point
SD	**Adr,Fs**	**Store double-precision Floating point**
MOVI2FP	Fd,Rs	Move 32 bits from integer registers to FP registers
MOVI2FP	Rd,Fs	Move 32 bits from FP registers to integer registers
MOVF	Fd,Fs	Copy one Floating point register to another register
MOVD	Dd,Ds	Copy a double-precision pair to another pair

Arithmetic/logic instructions in DLX

Opcode	*Operands*	*Meaning*
ADD	**Rd,Ra,Rb**	**Add**
ADDI	**Rd,Ra,Imm**	**Add immediate (all immediates are 16 bits)**
ADDU	Rd,Ra,Rb	Add unsigned
ADDUI	Rd,Ra,Imm	Add unsigned immediate
SUB	**Rd,Ra,Rb**	**Subtract**
SUBI	**Rd,Ra,Imm**	**Subtract immediate**
SUBU	Rd,Ra,Rb	Subtract unsigned
SUBUI	Rd,Ra,Imm	Subtract unsigned immediate
MULT	**Rd,Ra,Rb**	**Multiply signed**
MULTU	Rd,Ra,Rb	Multiply unsigned
DIV	**Rd,Ra,Rb**	**Divide signed**
DIVU	Rd,Ra,Rb	Divide unsigned
AND	**Rd,Ra,Rb**	**And**
ANDI	Rd,Ra,Imm	And immediate
OR	Rd,Ra,Rb	Or
ORI	Rd,Ra,Imm	Or immediate
XOR	Rd,Ra,Rb	Xor
XORI	Rd,Ra,Imm	Xor immediate
LHI	Rd,Imm	Load high immediate - loads upper half of register with immediate
SLL	Rd,Rs,Rc	Shift left logical
SRL	Rd,Rs,Rc	Shift right logical
SRA	Rd,Rs,Rc	Shift right arithmetic
SLLI	Rd,Rs,Imm	Shift left logical 'immediate' bits
SRLI	Rd,Rs,Imm	Shift right logical 'immediate' bits
SRAI	Rd,Rs,Imm	Shift right arithmetic 'immediate' bits
S__	**Rd,Ra,Rb**	**Set conditional: "__" may be EQ, NE, LT, GT, LE or GE**
S__I	Rd,Ra,Imm	Set conditional immediate: "__" may be EQ, NE, LT, GT, LE or GE
S__U	Rd,Ra,Rb	Set conditional unsigned: "__" may be EQ, NE, LT, GT, LE or GE
S__UI	Rd,Ra,Imm	Set conditional unsigned immediate: "__" may be EQ, NE, LT, GT, LE or GE
NOP	**-**	**No operation**

Control flow instructions in DLX

Opcode	*Operands*	*Meaning*
BEQZ	**Rt,Dest**	**Branch if Rt equal to zero; 16-bit offset from PC**
BNEZ	**Rt,Dest**	**Branch if Rt not equal to zero; 16-bit offset from PC**
BFPT	Dest	Test comparison bit in the FPSR (true) and branch; 16-bit offset from PC
BFPF	Dest	Test comparison bit in the FPSR (false) and branch; 16-bit offset from PC
J	**Dest**	**Jump: 26-bit offset from PC**
JR	**Rx**	**Jump: target in register**
JAL	**Dest**	**Jump and link: save PC+4 to R31; target is PC-relative**
JALR	Rx	Jump and link: save PC+4 to R31; target is a register
TRAP	Imm	Transfer to operating system at a vectored address; see next slide.
RFE	Dest	Return to user code from an exception

Floating-point instructions in DLX

Floating point instructions manipulate the FPR and indicate whether the operation is single or double precision.

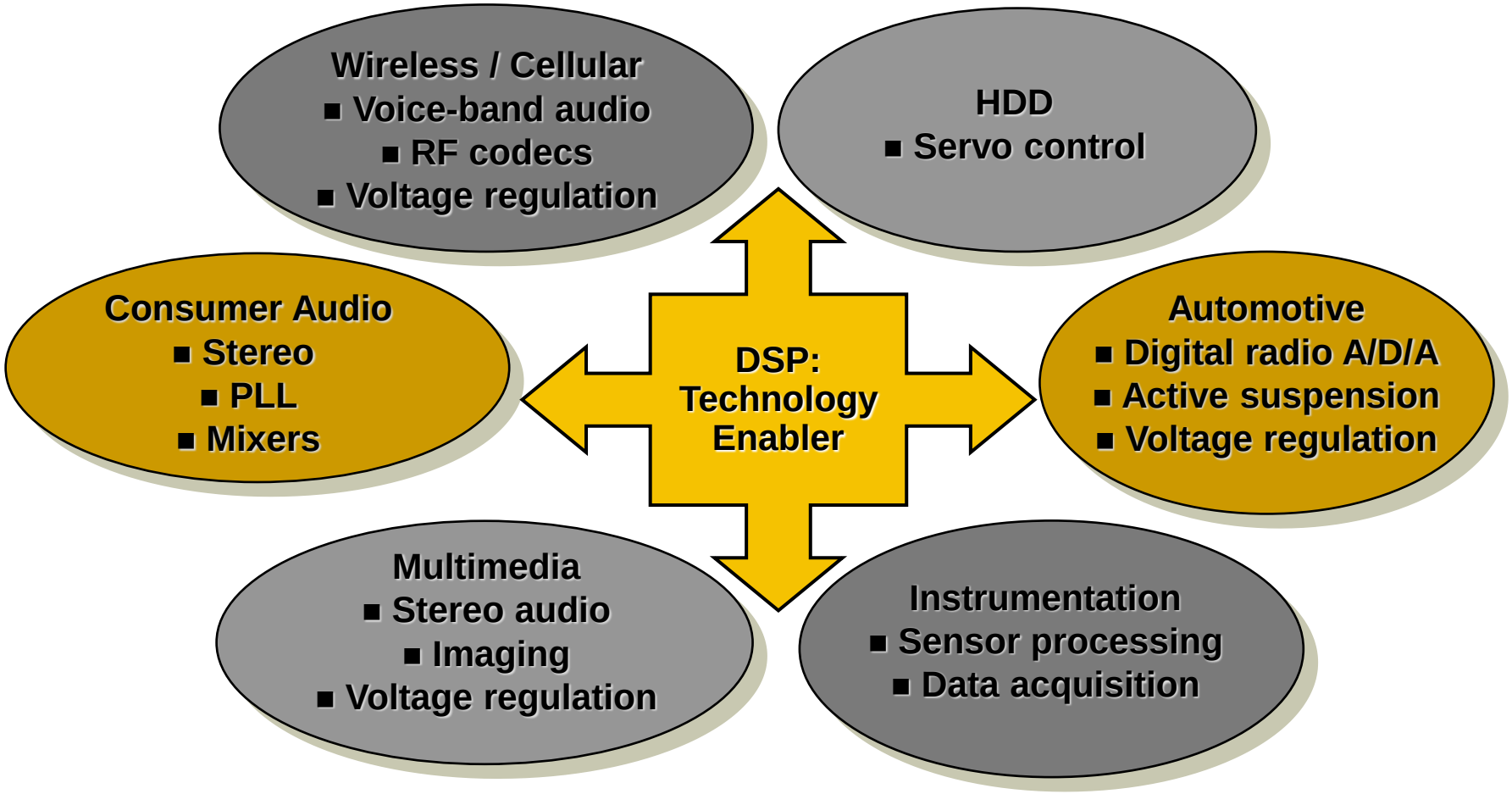
Single-precision operations can be applied to any of the registers, while double-precision ones apply only to even-odd pairs (e.g. F4,F5), which is designated by the even register number.

Opcode	*Operands*	*Meaning*
ADDD	**Dd,Da,Db**	**Add double-precision numbers**
ADDF	Fd,Fa,Fb	Add single-precision numbers
SUBD	**Dd,Da,Db**	**Subtract double-precision numbers**
SUBF	Fd,Fa,Fb	Subtract single-precision numbers
MULTD	**Dd,Da,Db**	**Multiply double-precision Floating point numbers**
MULTF	Fd,Fa,Fb	Multiply single-precision Floating point numbers
DIVD	**Dd,Da,Db**	**Divide double-precision Floating point numbers**
DIVF	Fd,Fa,Fb	Divide single-precision Floating point numbers
CVTF2D	Dd,Fs	Converts from type single-precision to type double-precision
CVTD2F	Fd,Ds	Converts from type double-precision to type single-precision
CVTF2I	Fd,Fs	Converts from type single-precision to type integer
CVTI2F	Fd,Fs	Converts from type integer to type single-precision
CVTD2I	Fd,Ds	Converts from type double-precision to type integer
CVTI2D	Dd,Fs	Converts from type integer to type double-precision
__D	Da,Db	Double-precision compares: "__" may be EQ, NE, LT, GT, LE or GE; sets comparison bit in FP status register
__F	Fa,Fb	Single-precision compares: "__" may be EQ, NE, LT, GT, LE or GE; sets comparison bit in FP status register

Specific ISA features in Digital Signal Processors (DSPs)

OVERVIEW

- In the late 1990s, digital signal applications, such as digital cellular telephones, emerged as one of the largest consumers of embedded computing power.
- Nowadays Digital Signal Processors (DSPs) are used in several embedded applications



SPECIFIC FEATURES OF DSPS AT ISA LEVEL

Compared to other embedded applications, DSP applications are differentiated by:

- Computationally demanding, numeric algorithms often composed of vector dot products
- Sensitivity to small numeric errors; for example, numeric errors may manifest themselves as audible noise in an audio device.
- Stringent real-time requirements.
- "Streaming" data; typically, input data is provided from an analog-to-digital converter as a infinite stream. Results are emitted in a similar fashion.
- High data bandwidth.
- Predictable, simple (though often eccentric) memory access patterns.
- Predictable program flow (typically characterized by nested loops).

In order to address this requirements DSPs are generally endowed with special features at ISA level, i.e.

- Highly-optimized instructions for arithmetic calculations (MAC, REPEAT)
- Complex addressing modes relying on special functional units and special memory architectures

SPECIAL DSP INSTRUCTIONS: MAC

The Sum of Products is the key element in most digital signal processing algorithms

Algorithm	Equation
Finite Impulse Response Filter	$y(n) = \sum_{k=0}^{M} a_k x(n-k)$
Infinite Impulse Response Filter	$y(n) = \sum_{k=0}^{M} a_k x(n-k) + \sum_{k=1}^{N} b_k y(n-k)$
Convolution	$y(n) = \sum_{k=0}^{N} x(k) h(n-k)$
Discrete Fourier Transform	$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-\frac{j2\pi nk}{N}}$
Discrete Cosine Transform	$F(u) = \sum_{x=0}^{N-1} c(u) \cdot f(x) \cdot \cos\left(\frac{\pi}{2N} u(2x+1)\right)$

In accordance with Amdahl's Law, in order to maximize performance, all DSPs are provided with a Multiply & Accumulate HW unit and a corresponding MAC instruction in the ISA

SPECIAL DSP INSTRUCTIONS: REPEAT

DSP applications require repeated execution of small number of arithmetic or multiplication instructions (*inner loops* or *kernels*)

In general, software looping uses a branch instruction to jump back to the start of kernel fixed number of times

Loop counter must be maintained

- Extra register is required for loop counter

Loop counter must be incremented

- Data path is used for incrementing, thus normal data processing can not be utilized on those cycles

Loop counter must be tested against the end value and branch may require several cycles to complete

Waste of clock cycles for small kernels is not acceptable

REPEAT INSTRUCTION: EXAMPLE

Software looping takes roughly three times longer to execute than hardware looping using a loop instruction

```
;SW LOOPING
MOVE #16,B
LOOP: MAC (R0)+,(R4)+, A
DEC B
JNE LOOP

;HW LOOPING
RPT #16
MAC (R0)+,(R4)+,A
```

Hardware looping **(*zero-overhead looping*) is utilized to avoid waste of clock cycles (*overhead*)**

Loop counter increment, test against end condition and branching are done by hardware instead of by software

ADDRESSING AND MEMORY REQUIREMENTS

As many DSP algorithms access large amounts of data (usually stored in arrays) in a very streamlined way, DSPs usually need:

- The ability to perform multiple memory accesses within an instruction cycle
- The ability to perform a fast new address calculation (especially with auto-increment, auto-decrement capabilities), while performing other arithmetic operations in parallel

A Harvard, super-Harvard (or similar) dual memory architecture

A large number of flexible register indirect addressing modes (with auto-decrement, auto-increment, indexing or displacement options)

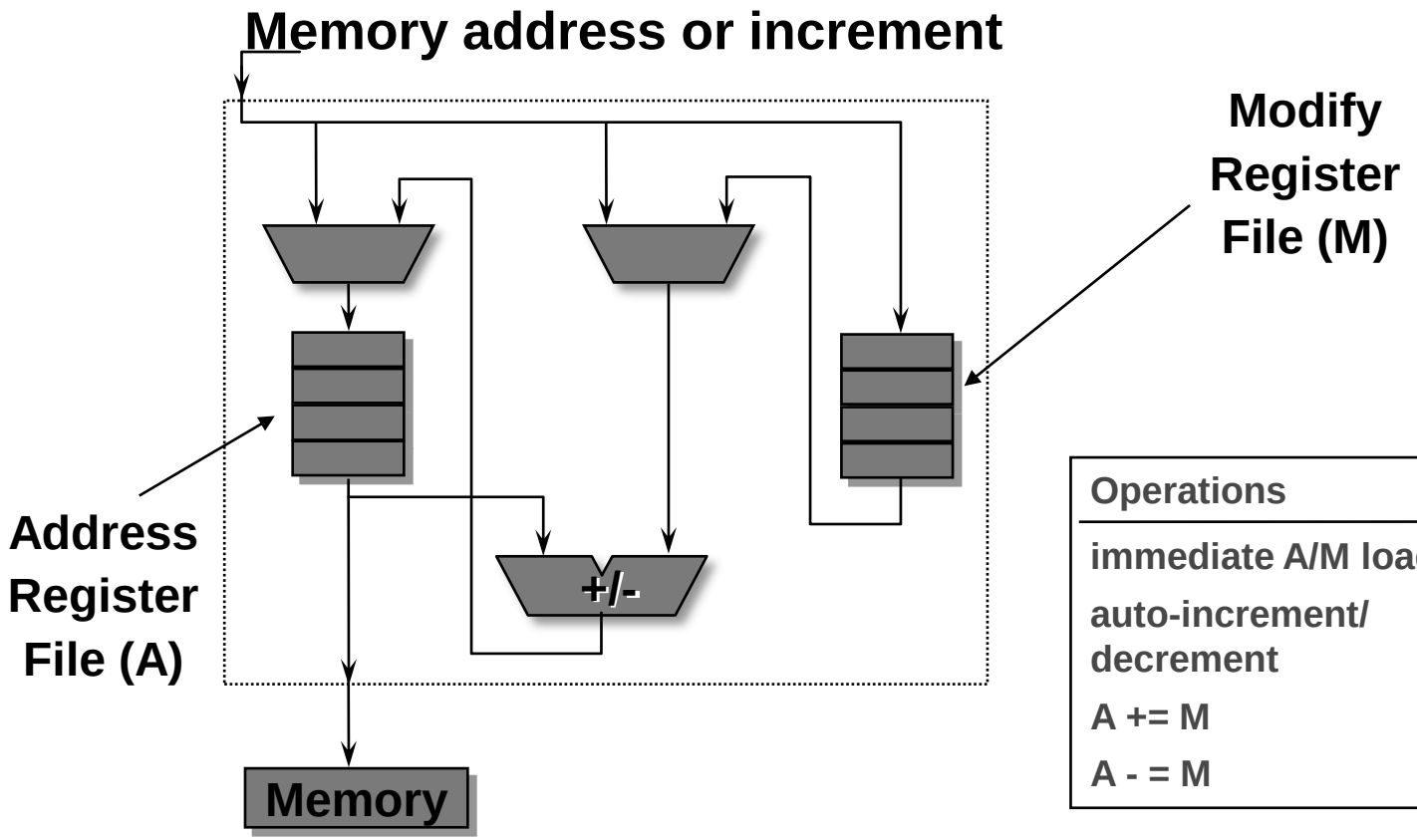
Example. **In DSP32xx address register R5 is post-incremented with value stored in register R17 as follows A0 = A0 + *R5++R17;**

A special functional unit explicitly devoted to calculating data addresses (Address Generation Unit or AGU)

THE ADDRESS GENERATION UNIT (AGU)

Address generation unit (AGU) consists of a specific ALU and a separate register file to calculate memory addresses

In this way, address generation for subsequent instructions can be executed in parallel



Operations	cost (clock cycles)
immediate A/M load	1
auto-increment/ decrement	0
A += M	0
A - = M	0

SPECIAL ADDRESSING MODES: CIRCULAR

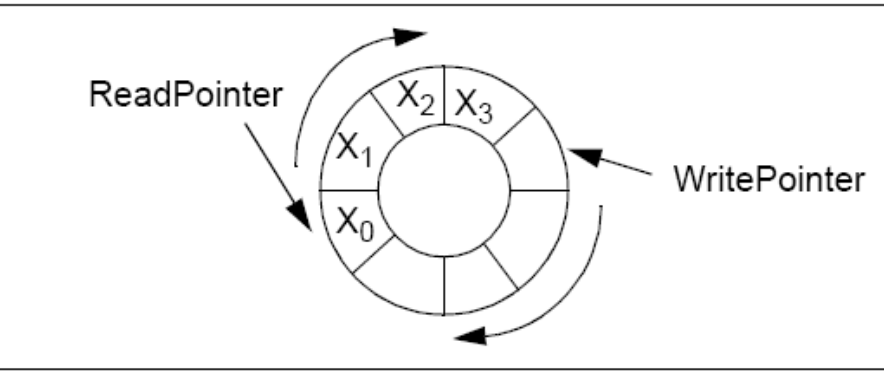
DSPs deal with iterative computation on continuous sets of data ⇨ FIFO buffers and circular buffers are used to store data in adjacent locations

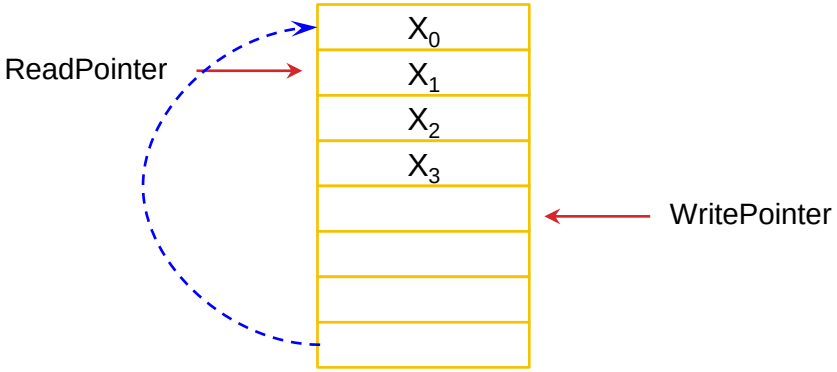
Usually the programmer needs to maintain two pointers:

- read pointer: address of memory location to be read next
- write pointer: address of memory location where the next data value is to be written

Each time a read or write operation is performed, the programmer needs to check whether the end of the buffer has been reached

At the end of the buffer, the pointer is set to point to the beginning





SPECIAL ADDRESSING MODES: CIRCULAR

Modulo addressing (*circular addressing*) provides hardware support for checking the end of the buffer

AGU address registers *Rx*:

- are updated with pre- or post-increment *k*
- address generation is based on modulo arithmetic

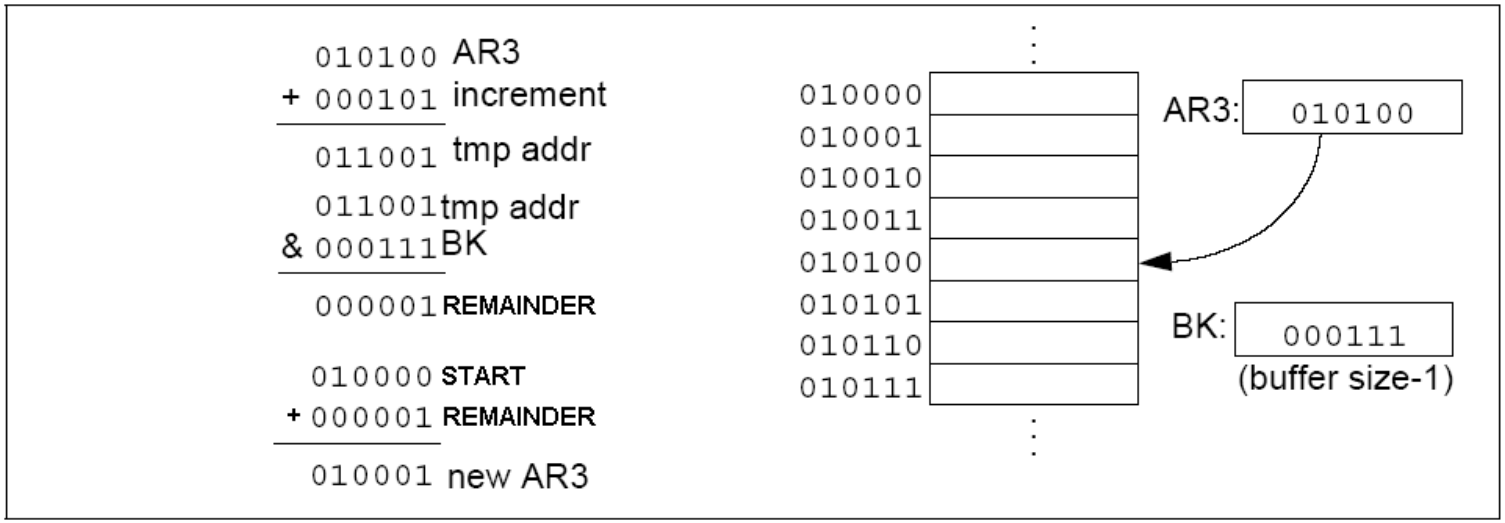
Programmer needs to store the length of the circular buffer into a special *modifier Mx* or *modulo register* of the AGU

- Each modifier register is associated with one or more address registers
- Starting address of the buffer is not specified; address register must be initialized with a valid value before usage
- Circular buffers usually begin at *k*-word boundaries, where *k* is smallest power of two that is equal to or greater than the size of the buffer, for instance a 48-word circular buffer must reside in 64-word boundary, i.e. starting address may be 0, 64, 128, 192 etc…

EXAMPLE

In TMS320C30 the circular addressing is denoted by %. Let us assume that the buffer size is initialized earlier to 8 (register BK) and currently register AR3 is pointing to address 010100, i.e. the buffer starts at 010000 (8 word boundary).

What is the contents of AR3 after instruction which has addressing field: A0 = A0 + *++AR3(5)% ?



SPECIAL ADDRESSING MODES: BIT-REVERSED

"Bit reversed" addressing mode: Address values are reversed bit-by-bit

Targeted to Fast Fourier Transform. The output of an FFT algorithm is usually bit-reversed, i.e., the data points are produced out of sequence:

0 (000)	=>	0 (000)
1 (001)	=>	4 (100)
2 (010)	=>	2 (010)
3 (011)	=>	6 (110)
4 (100)	=>	1 (001)
5 (101)	=>	5 (101)
6 (110)	=>	3 (011)
7 (111)	=>	7 (111)

Hardware implementation may be:

- real bit reversal between address register and address bus
- reverse-carry arithmetic in AGU

EXAMPLE

In TMS320C3X bit-reversed addressing mode notation is symbol "B".

Let us suppose the data be stored in memory starting from address 60h (= AR2) and the length of FFT is 8 (IR0 contains 100):

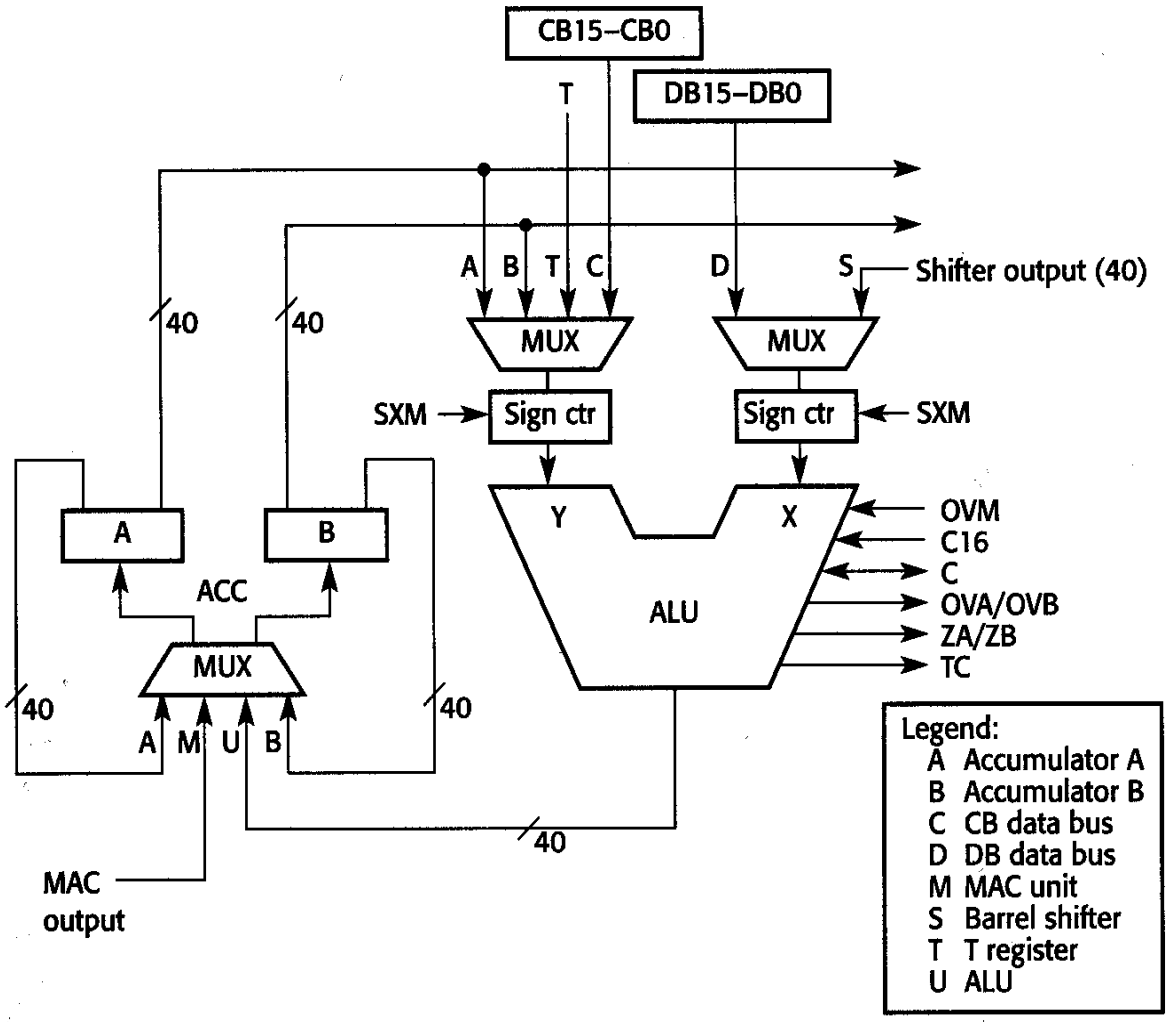
```
*AR2++(IR0)B ; AR2 = 0110 0000 = 60 (0. sample)
*AR2++(IR0)B ; AR2 = 0110 0100 = 64 (1. sample)
*AR2++(IR0)B ; AR2 = 0110 0010 = 62 (2. sample)
*AR2++(IR0)B ; AR2 = 0110 0110 = 66 (3. sample)
*AR2++(IR0)B ; AR2 = 0110 0001 = 61 (4. sample)
*AR2++(IR0)B ; AR2 = 0110 0101 = 65 (5. sample)
*AR2++(IR0)B ; AR2 = 0110 0011 = 63 (6. sample)
*AR2++(IR0)B ; AR2 = 0110 0111 = 67 (7. sample)
```

Reverse order addition and carry propagation

PUTTING ALL TOGETHER… THE TI TMS320C54 DSP

MAIN FEATURES

- Dual-access memory through 2 16-bit buses (CB,DB)
- 40-bit ALU
- 16x16 multiplier
- 2 40-bits accumulators
- Most operand combinations are possible at ALU input through appropriate MUX



FREQUENCY OF INSTRUCTION CLASSES

As in previous examples, simple instructions dominate this list of most frequent instructions.

The 16 suffix stands for a 16-bit instruction operand, whereas 32 stands for a 32-bit operand.

The large number of change status instructions is to set mode bits to affect instructions, essentially saving opcode space in these 16-bit instructions by keeping some of it in a status register.

These data were collected from a measurement of static instructions for the C-callable library of 54 DSP routines coded in assembly language.

Instruction	Percent
store mem16	32.2%
load mem16	9.4%
add mem16	6.8%
call	5.0%
push mem16	5.0%
subtract mem16	4.9%
multiple-accumulate (MAC) mem16	4.6%
move mem-mem 16	4.0%
change status	3.7%
pop mem16	2.8%
conditional branch	2.6%
load mem32	2.5%
return	2.5%
store mem32	2.0%
branch	2.0%
repeat	2.0%
multiply	1.8%
NOP	1.5%
add mem32	1.3%
subtract mem32	0.9%
Total	97.2%

FREQUENCY OF ADDRESSING MODES

Addressing Mode	Assembly Symbol	Percent
Immediate	#num	30.02%
Displacement	ARx(num)	10.82%
Register indirect	*ARx	17.42%
Direct	num	11.99%
Autoincrement, pre increment (increment register *before* use contents as address)	*+ARx	0
Autoincrement, post increment (increment register *after* use contents as address)	*ARx+	18.84%
Autoincrement, pre increment with 16b immediate	*+ARx(num)	0.77%
Autoincrement, pre increment, with circular addressing	*ARx+%	0.08%
Autoincrement, post increment with 16b immediate, with circular addressing	*ARx+(num)%	0
Autoincrement, post increment by contents of AR0	*ARx+0	1.54%
Autoincrement, post increment by contents of AR0, with circular addressing	*ARx+0%	2.15%
Autoincrement, post increment by contents of AR0, with bit reverse addressing	*ARx+0B	0
Autodecrement, post decrement (decrement register *after* use contents as address	*ARx-	6.08%
Autodecrement, post decrement, with circular addressing	*ARx-%	0.04%
Autodecrement, post decrement by contents of AR0	*ARx-0	0.16%
Autodecrement, post decrement by contents of AR0, with circular addressing	*ARx-0%	0.08%
Autodecrement, post decrement by contents of AR0, with bit reverse addressing	*ARx-0B	0
Total		100.00%

The C54x has 17 data addressing modes, but the first 4 basic modes common to all RISC architectures account for 70% of the total.

Auto-increment and auto-decrement modes instead, account for another 25%.

This data was collected from a measurement of static instructions for the C-callable library of 54 DSP routines coded in assembly language.

AN ASSEMBLY PROGRAM EXAMPLE

```
;sum elements of a table                ; this is a comment
.data                                   ; start data section
table     .int   2,4,6,8,10,12          ; defines a table of integer values referred to as table
result    .int   0                      ; defines a local integer variable referred to as result
          .eval (result-table), lgth    ; evaluates result (address)-table(address)=lenght
.text                                   ; start text section
START     RESET                         ; reset the processor
          LD        #lgth, B            ; load lght in B
          LD        #0000, A            ; initialize A with constant 0
          STM       #table, AR3         ; initialize AR3 with table address
          STM       #result, AR4        ; initialize AR4 with result address
LOOP      ADD       *AR3+, A            ; accumulate the value pointed by AR3 in A
          SUB       #0001, B            ; decrement control variable in B by 1
          BC        LOOP, BNEQ          ; conditional jump. Jump if result is different from 0
          STL       A, *AR4             ; store final result
          .end                          ; terminates assembly. All instructions after this
                                        ; directive are ignored
```

MAIN REFERENCES

H. Hennessy, D. A. Patterson, "Computer Architecture: a quantitative approach," *Morgan Kaufmann*, 2002, Chapter 2.

P. Lapsley, J. Bier, A. Shoam, E. A. Lee, "DSP processor fundamentals: Architectures and Features", *IEEE, Wiley-Interscience*, 1996, Chapter 6,7.

A. Bateman, I. Paterson-Stephens, "The DSP Handbook", Algorithms, Applications and Design Techniques, *Prentice Hall*, 2002, Chapter 2.